



Web Programming
Lecture 13

React - Redux Toolkit

Mahhek Tahir



Redux

Redux is a **state management library** for JavaScript applications, mainly used with **React**.

It helps you manage and **store data in a central place** instead of passing it between components manually.

Think of it like a storage box where all your app's data is kept in one place, so every part of your app can access it easily.



Why do we need Redux?

Without Redux (Normal React State)

- Each component **manages its own state** using `useState()`.
- Passing data between components requires **props drilling** (passing props from parent → child → child).
- Managing complex states like user authentication, or shopping carts.

With Redux

- Redux **stores all the state in a single place (Store)**.
- Any component can **access or update the state easily**.
- **No need for props drilling!**



Redux Toolkit

Redux **Toolkit** (RTK) is the **official way** to write Redux code **easier and faster**.

It reduces **boilerplate code** and makes Redux **simpler to use**.

Traditional Redux → Too much setup (Actions, Reducers, Store separately)

Redux Toolkit → All in one place (**createSlice()**)

- `npm install @reduxjs/toolkit react-redux`

Key concepts of Redux Toolkit



Store (Central place for state)

- The **global state** of your app.
- Components **get data from** and **send updates to** the store.

Actions (Tell Redux what to do)

- Actions are **messages** that tell Redux what **changes should happen**

Reducers (Update the state)

- **Pure Functions** that **modify the store** when actions are dispatched.
- In RTK, reducers are inside **createSlice()**

Redux Toolkit we write actions inside the **reducers object**, but technically we are writing reducer functions, and **Redux Toolkit automatically creates the actions for us.**

Dispatch (Send actions to update the state)

- Used in components to **trigger actions**.

Selectors (Read data from the store)

- Used in components to **get data from Redux** (**useSelector()**).



Example

Store → A big container that stores all the state (data).

Selectors → Functions that **extract** specific data from the store.

Actions → Instructions that **tell Redux what to change** (like “increase the counter”).

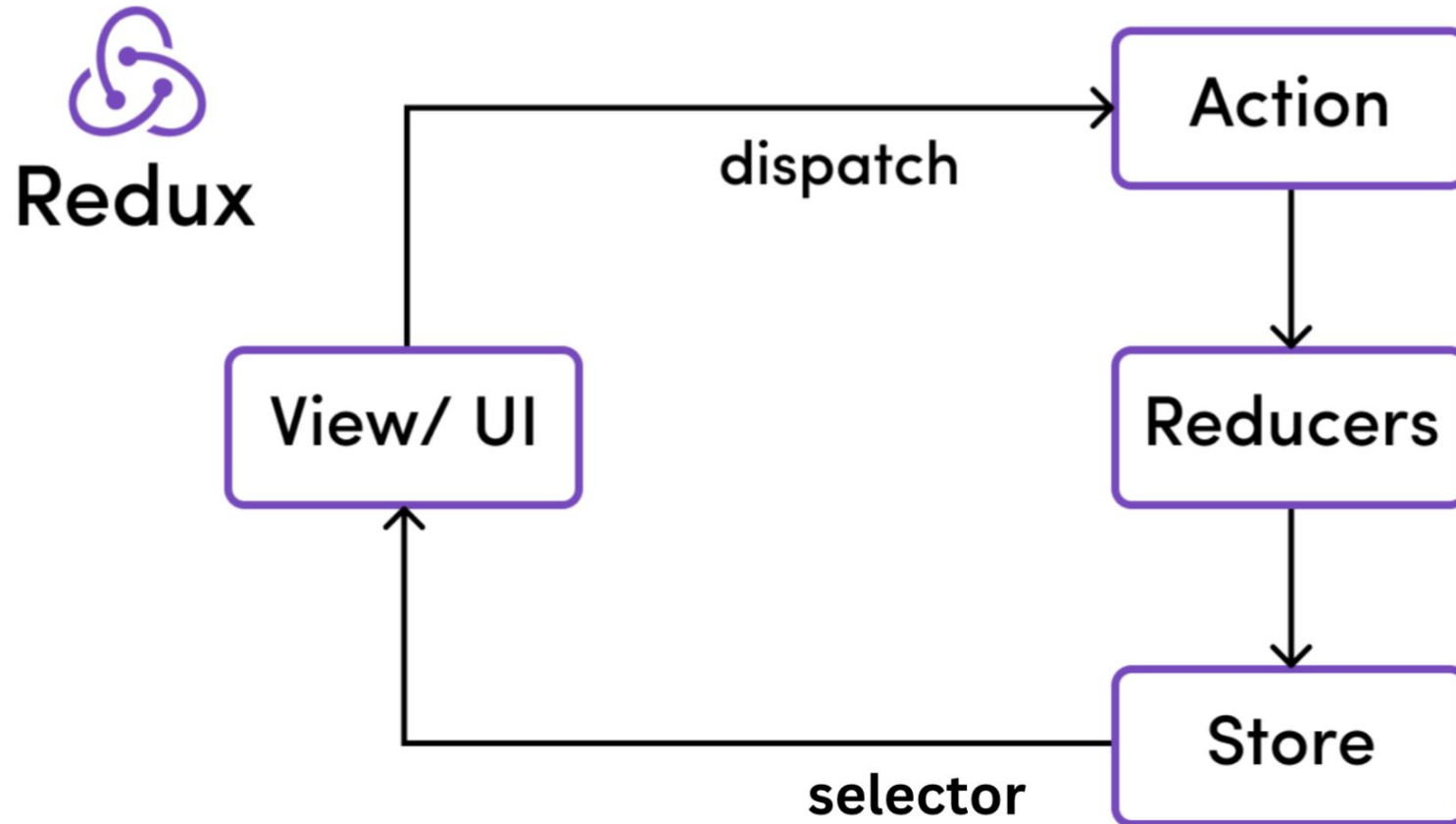
Dispatcher → **Triggers** actions and tells Redux, "Hey, do this now!"

Reducers → The actual **workers** that **update the state** based on the action.

Example: If you click a "Like" button in an app:

- **Action** → "Like Post"
- **Dispatcher** → Sends "Like Post" to Redux
- **Reducer** → Increases the like count in the store
- **Selector** → Gets the updated like count to show on the screen

Diagram





Slice (action + reducer)

A **slice** in Redux Toolkit is like a **state variable** but with built-in actions and an initial value.

With `createSlice()`, **reducers and actions are defined together** inside a single slice.

Think of this like

```
const [counter, setCounter] = useState(0);
```

but for **global state**, where `counter` also has actions like `increment` and `decrement`.

Redux Slice = State + Actions in One Place

When you create a slice:

1. **You define a state variable** (`counter` with an initial value).
2. **You define actions** (`increment`, `decrement`).
3. **Redux handles updating the state for you.**



Slice Example (counterSlice.js)

```
import { createSlice } from "@reduxjs/toolkit";

const counterSlice = createSlice({
  name: "counter", // <-- This is where "counter" is first set!
  initialState: { value: 0 }, // <-- Initial Value
  reducers: {
    increment: (state) => { state.value += 1; },
    decrement: (state) => { state.value -= 1; },
    incrementByAmount: (state, action) => { state.value += action.payload; },
  },
});

// Export actions
export const { increment, decrement, incrementByAmount } = counterSlice.actions;

// Export reducer
export default counterSlice.reducer;
```

Store



The **Redux store** is like a **central database** for your app. It holds all the **global state** and allows components to **read data** or **update the state** by dispatching actions.

Holds the entire state of the application

Allows access to state via `useSelector`

Allows state updates via `dispatch(action)`

Uses reducers to modify the state



Registering the counterSlice

```
// store.js
import { configureStore } from "@reduxjs/toolkit";
import counterReducer from "../counterSlice";

export const store = configureStore({
  reducer: {
    counter: counterReducer
  }
});
```



Provide Store to App

```
// main.jsx or index.js
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";
import { Provider } from "react-redux";
import { store } from "./store";

ReactDOM.createRoot(document.getElementById("root")).render(
  <Provider store={store}>
    <App />
  </Provider>
);
```



Selector

A **selector** is a function used to **retrieve** data from the Redux store. It helps in **reading** and using the state inside React components.

```
const count = useSelector((state) =>
state.counter.value); // Selecting counter value

const user = useSelector((state) => state.user); //
Selecting user object
```

useSelector takes a function that returns **a part of the Redux state**.

Avoid **directly selecting the whole store**, pick only needed values.



Dispatcher

A **dispatcher** is a function that **sends actions to the Redux store** to update state.

We use `useDispatch` from Redux Toolkit to trigger actions.

```
return (  
  <div>  
    <button onClick={() => dispatch(increment())}>+  
Increment</button>  
    <button onClick={() => dispatch(decrement())}>-  
Decrement</button>  
  </div>  
);
```

`dispatch(action())` **calls an action** that updates the state.

```
import { useSelector, useDispatch } from "react-redux";
import { increment, decrement, incrementByAmount } from "../counterSlice";

function Counter() {

  const count = useSelector((state) => state.counter.value); // selector

  const dispatch = useDispatch(); // dispatcher

  return (
    <div>
      <h2>Count: {count}</h2>

      <button onClick={() => dispatch(increment())}>
        Increment
      </button>

      <button onClick={() => dispatch(decrement())}>
        Decrement
      </button>

      <button onClick={() => dispatch(incrementByAmount(5))}>
        Add 5
      </button>
    </div>
  );
}

export default Counter;
```



2 slices with same names (alias)

```
import { increment as counterIncrement } from "./counterSlice";  
import { increment as cartIncrement } from "./cartSlice";
```




```
const userSlice = createSlice({
  name: "user",
  initialState: {
    name: "",
    age: null,
  },
  reducers: {
    setName: (state, action) => {
      state.name = action.payload;
    },
    setAge: (state, action) => {
      state.age = action.payload;
    },
    resetUser: (state) => {
      state.name = "";
      state.age = null;
    }
  }
});
```

```
export const { setName, setAge, resetUser } = userSlice.actions;
export default userSlice.reducer;
```

Store.js

```
import { configureStore } from "@reduxjs/toolkit";
import counterReducer from "../counterSlice";
import userReducer from "../userSlice"; // Import both reducers

const store = configureStore({
  reducer: {
    counter: counterReducer, // Assign counter slice
    user: userReducer, // Assign user slice
  },
});

export default store;
```

Selector + Dispatcher in App.js



```
import { useSelector, useDispatch } from "react-redux";
import { increment, decrement } from "./counterSlice";
import { setName, setAge } from "./userSlice";

const App = () => {
  // ♦ Use `useSelector` to access store data
  const count = useSelector((state) => state.counter.value);
  const user = useSelector((state) => state.user);

  // ♦ Use `useDispatch` to call actions
  const dispatch = useDispatch();

  return (
    <div>
      <h1>Counter: {count}</h1>
      <button onClick={() => dispatch(increment())}>+ Increment</button>
      <button onClick={() => dispatch(decrement())}>- Decrement</button>

      <h2>User: {user.name} (Age: {user.age})</h2>
      <button onClick={() => dispatch(setName("Alice"))}>Set Name: Alice</button>
      <button onClick={() => dispatch(setAge(30))}>Set Age: 30</button>
    </div>
  );
};
```





Selector in Profile Component

```
import { useSelector } from "react-redux";

const Profile = () => {
  // ♦ Get `count` and `user` from the Redux store
  const count = useSelector((state) => state.counter.value);
  const user = useSelector((state) => state.user);

  return (
    <div>
      <h2>🔑 Profile Component</h2>
      <p>Counter Value: {count}</p>
      <p>User Name: {user.name}</p>
      <p>User Age: {user.age}</p>
    </div>
  );
};

export default Profile;
```

Slice with parameters



```
import { createSlice } from "@reduxjs/toolkit";

const counterSlice = createSlice({
  name: "counter",
  initialState: { value: 0 },
  reducers: {
    increment: (state) => { state.value += 1; },
    decrement: (state) => { state.value -= 1; },
    incrementByAmount: (state, action) => {
      state.value += action.payload; // Using parameter from action
    }
  }
});

export const { increment, decrement, incrementByAmount } = counterSlice.actions;
export default counterSlice.reducer;
```

```
<button onClick={() => dispatch(incrementByAmount(5))}>Increment by 5</button>
```



Slice with multiple parameters

```
const userSlice = createSlice({  
  name: "user",  
  initialState: { name: "", age: 0 },  
  reducers: {  
    updateUser: (state, action) => {  
      state.name = action.payload.name;  
      state.age = action.payload.age;  
    }  
  }  
});
```

```
dispatch(updateUser({ name: "Alice", age: 30 }));
```


Add tasks - array params



```
import { createSlice } from "@reduxjs/toolkit";
```

```
const tasksSlice = createSlice({  
  name: "tasks",  
  initialState: { tasks: [] }, // Array to store tasks  
  reducers: {  
    addTasks: (state, action) => {  
      state.tasks = [...state.tasks, ...action.payload]; // Merge new tasks  
    }  
  }  
});
```

```
export const { addTasks } = tasksSlice.actions;  
export default tasksSlice.reducer;
```

```
const newTasks = ["Buy groceries", "Complete project", "Read a book"];  
dispatch(addTasks(newTasks));
```